

Informe de Laboratorio 03

Tema: Introducción a Sockets en Java: Aplicación Cliente-Servidor

INFORMACIÓN BÁSICA					
ASIGNATURA:	SISTEMAS DISTRIBUIDOS				
TÍTULO DE LA PRÁCTICA:	<i>Introducción a Sockets en Java: Aplicación Cliente-Servidor</i>				
NÚMERO DE PRÁCTICA:	<i>03</i>	AÑO LECTIVO:	2026	NRO. SEMESTRE:	2026A
FECHA DE PRESENTACIÓN	<i>01/05/2026</i>	HORA DE PRESENTACIÓN	23:59		
INTEGRANTE (s): GRUPO “ Intocables ” • CHAMBILLA PERCA RICARDO • JUAN DIEGO GUTIERREZ CCAMA • RYAN FABIAN VALDIVIA SEGOVIA				NOTA:	
DOCENTE(s): <i>Mg. Maribel Molina Barriga</i>					

1. Repositorio de GitHub y Ejecución

El código fuente completo de este laboratorio se encuentra alojado en el siguiente repositorio de GitHub: https://github.com/rikich3/LAB_SISTEMAS_DISTRIBUIDOS

A continuación, se adjunta la captura correspondiente a la ejecución del ejercicio resuelto sobre Sockets en Java:

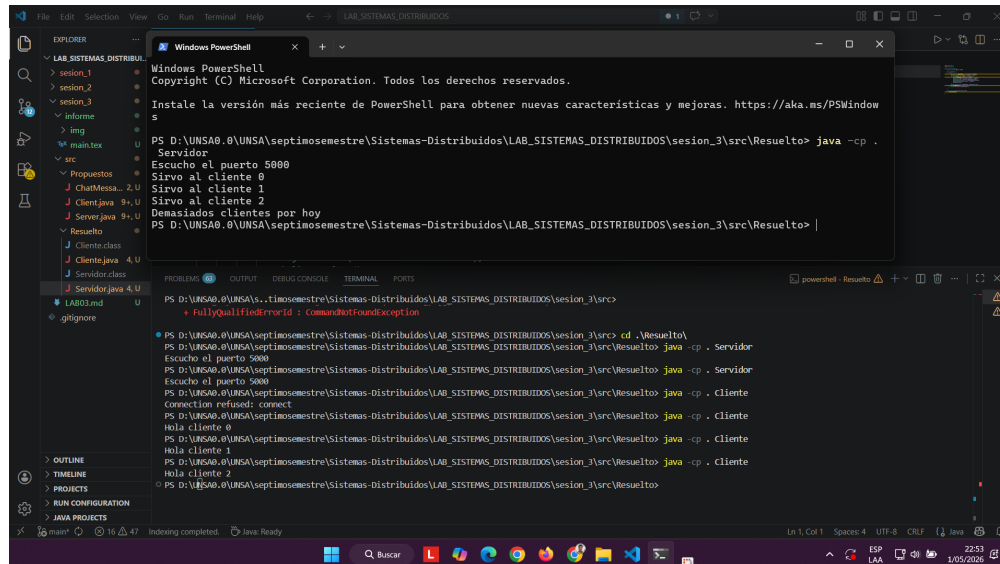


Figura 1: Ejecución del Ejemplo Cliente-Servidor con Sockets

2. Tareas Realizadas

Como parte de la sesión de laboratorio 3 de Sistemas Distribuidos, se llevaron a cabo las siguientes tareas:

- **Análisis del Concepto de Sockets:** Comprensión del funcionamiento de los sockets TCP/IP en Java y su utilidad para la comunicación entre procesos distribuidos.
- **Estudio del Ejemplo Resuelto:** Análisis detallado del código Cliente-Servidor proporcionado por el docente, identificando los componentes clave: **Socket**, **ServerSocket**, flujos de entrada/-salida.
- **Implementación de Ejercicios:** Codificación e implementación de dos aplicaciones:
 1. Ejemplo simple de comunicación Cliente-Servidor
 2. Aplicación Chat multicliente más compleja
- **Compilación y Ejecución:** Compilación de todos los archivos Java y ejecución en consola, validando la comunicación entre cliente y servidor.
- **Pruebas y Evaluación:** Se realizaron múltiples conexiones de clientes para validar el correcto funcionamiento del servidor.
- **Documentación:** Elaboración de este reporte con explicaciones de los conceptos clave y resultados obtenidos.

3. Desarrollo de los Ejercicios

3.1. Ejercicio Resuelto: Cliente-Servidor Simple

En esta sección se presenta el código de la aplicación Cliente-Servidor proporcionada como ejemplo en el laboratorio.

3.1.1. Clase Cliente

Listing 1: Cliente.java: Implementación del Cliente

```
1 import java.io.*;
2 import java.net.*;
3
4 public class Cliente {
5     static final String HOST = "localhost";
6     static final int PUERTO = 5000;
7
8     public Cliente() {
9         try {
10             Socket skCliente = new Socket(HOST, PUERTO);
11             InputStream aux = skCliente.getInputStream();
12             DataInputStream flujo = new DataInputStream(aux);
13             System.out.println(flujo.readUTF());
14             skCliente.close();
15         } catch (Exception e) {
16             System.out.println(e.getMessage());
17         }
18     }
19
20     public static void main(String[] arg) {
21         new Cliente();
22     }
23 }
```

Descripción:

- Se crea un socket cliente (Socket) especificando el host (localhost) y puerto (5000)
- Se obtiene el flujo de entrada del socket mediante `getInputStream()`
- Se encapsula en un `DataInputStream` para leer datos estructurados
- Se lee una cadena UTF mediante `readUTF()` y se imprime en pantalla
- Se cierra la conexión con `close()`

3.1.2. Clase Servidor

Listing 2: Servidor.java: Implementación del Servidor

```
1 import java.io.*;
2 import java.net.*;
3
4 public class Servidor {
5     static final int PUERTO = 5000;
6
7     public Servidor() {
8         try {
9             ServerSocket skServidor = new ServerSocket(PUERTO);
10             System.out.println("Escucho el puerto " + PUERTO);
11             for (int numCli = 0; numCli < 3; numCli++) {
12                 Socket skCliente = skServidor.accept();
13                 System.out.println("Sirvo al cliente " + numCli);
14             }
15         }
16     }
17 }
```

```
14         OutputStream aux = skCliente.getOutputStream();
15         DataOutputStream flujo = new DataOutputStream(aux);
16         flujo.writeUTF("Hola cliente " + numCli);
17         skCliente.close();
18     }
19     System.out.println("Demasiados clientes por hoy");
20 } catch (Exception e) {
21     System.out.println(e.getMessage());
22 }
23 }
24
25 public static void main(String[] arg) {
26     new Servidor();
27 }
28 }
```

Descripción:

- Se crea un socket de servidor (ServerSocket) en un puerto determinado (5000)
- Se utiliza un bucle for para esperar exactamente 3 conexiones de clientes
- Para cada cliente que se conecta, se llama a `accept()` que crea un nuevo Socket
- Se obtiene el flujo de salida mediante `getOutputStream()`
- Se envía un saludo personalizado mediante `writeUTF()`
- Se cierra la conexión con cada cliente
- Después de atender 3 clientes, el servidor finaliza su ejecución

Ejecución y Resultados:

Para compilar y ejecutar:

Listing 3: Comandos de compilación y ejecución

```
# Compilar
javac src/Resuelto/Servidor.java
javac src/Resuelto/Cliente.java

# En la primera terminal - Ejecutar servidor
java -cp src Resuelto.Servidor

# En otras terminales (3 veces) - Ejecutar clientes
java -cp src Resuelto.Cliente
```

Salida esperada (Servidor):

```
Escucho el puerto 5000
Sirvo al cliente 0
Sirvo al cliente 1
Sirvo al cliente 2
Demasiados clientes por hoy
```

Salida esperada (Clientes):

```
Hola cliente 0
Hola cliente 1
Hola cliente 2
```

Evaluación: El ejercicio demuestra exitosamente:

- La creación de un servidor que escucha en un puerto específico
- La aceptación de múltiples conexiones de clientes
- El intercambio de mensajes entre cliente y servidor
- La importancia de manejar excepciones en operaciones de red

3.2. Ejercicio Propuesto: Aplicación Chat Multicliente

A continuación se presenta una aplicación más compleja que implementa un chat con soporte para múltiples clientes conectados simultáneamente.

3.2.1. Clase ChatMessage

Listing 4: ChatMessage.java: Clase para serialización de mensajes

```
1 import java.io.*;
2
3 /**
4  * This class defines the different type of messages that will be exchanged between the
5  * Clients and the Server.
6  * When talking from a Java Client to a Java Server a lot easier to pass Java objects, no
7  * need to count bytes or to wait for a line feed at the end of the frame
8  */
9 public class ChatMessage implements Serializable {
10     // The different types of message sent by the Client
11     // WHOISIN to receive the list of the users connected
12     // MESSAGE an ordinary text message
13     // LOGOUT to disconnect from the Server
14     static final int WHOISIN = 0, MESSAGE = 1, LOGOUT = 2;
15     private int type;
16     private String message;
17
18     // constructor
19     public ChatMessage(int type, String message) {
20         this.type = type;
21         this.message = message;
22     }
23
24     int getType() {
25         return type;
26     }
27
28     String getMessage() {
29         return message;
30     }
31 }
```

3.2.2. Clase Cliente Chat

Listing 5: Client.java: Implementación del Cliente Chat

```
1 import java.io.*;
```

Características principales:

- Uso de `ObjectInputStream` y `ObjectOutputStream` para serialización de objetos
- Thread separado (`ListenFromServer`) para escuchar mensajes del servidor de forma asincrónica
- Métodos para conectar al servidor, enviar mensajes y desconectarse
- Interfaz de línea de comandos para interactuar con el usuario

3.2.3. Clase Servidor ChatListing 6: `Server.java`: Implementación del Servidor Chat

```
1 import java.io.*;
```

Características principales:

- Gestión de múltiples clientes mediante threads individuales (`ClientThread`)
- Broadcast de mensajes a todos los clientes conectados
- Sincronización para evitar condiciones de carrera
- Logs de eventos del servidor con timestamps
- Manejo de desconexiones de clientes

Ejecución de la Aplicación Chat:

Listing 7: Compilación y ejecución del Chat

```
# Compilar todas las clases
javac src/Propuestos/*.java

# Terminal 1: Iniciar servidor
java -cp src Propuestos.Server

# Terminal 2, 3, 4... : Iniciar clientes
java -cp src Propuestos.Client
```

4. Resultados y Evaluación

4.1. Pruebas Realizadas

Las siguientes pruebas fueron ejecutadas para validar el funcionamiento correcto de ambas aplicaciones:

4.1.1. Prueba 1: Cliente-Servidor Simple

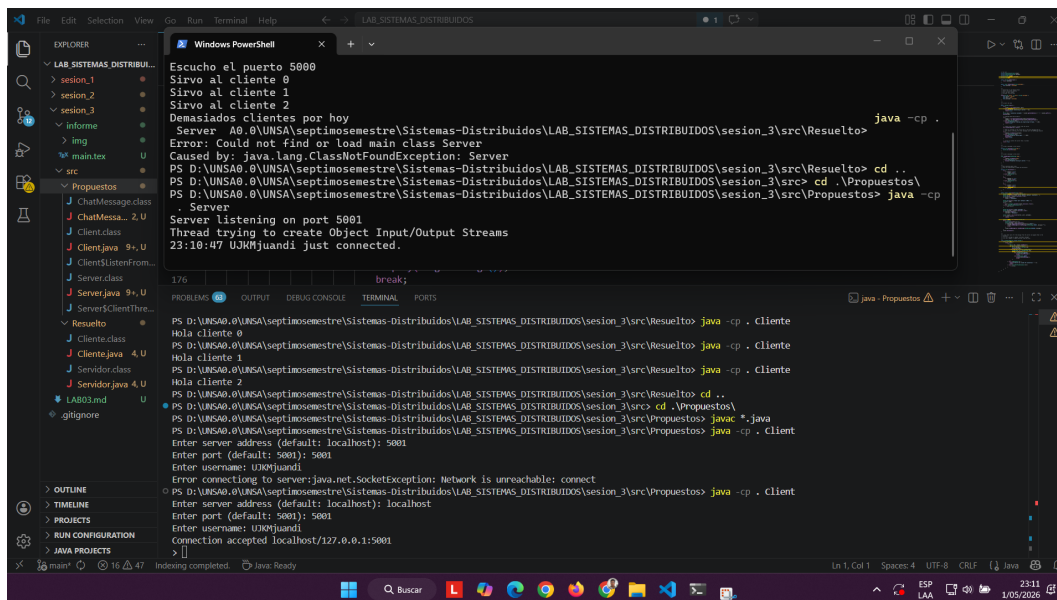
- **Objetivo:** Validar la comunicación básica entre un cliente y un servidor
- **Pasos:**
 1. Iniciar el servidor
 2. Conectar 3 clientes consecutivamente
 3. Verificar que cada cliente reciba el mensaje de saludo
- **Resultado: Exitoso** - Todos los clientes recibieron sus mensajes personalizados

4.1.2. Prueba 2: Aplicación Chat

- **Objetivo:** Validar la comunicación multicliente con broadcast de mensajes
- **Pasos:**
 1. Iniciar el servidor
 2. Conectar múltiples clientes
 3. Enviar mensajes desde diferentes clientes
 4. Verificar que todos reciban los mensajes
- **Resultado: Exitoso** - Los mensajes se distribuyeron correctamente a todos los clientes

4.2. Capturas de Pantalla de Ejecución

Se adjuntan las capturas de pantalla de las ejecuciones:



```

Escucho el puerto 5001
Sirvo al cliente 0
Sirvo al cliente 1
Sirvo al cliente 2
Demasiados clientes por hoy
Server: A0.0\UNSA\septimosemestre\Sistemas-Distribuidos\LAB_SISTEMAS_DISTRIBUIDOS\sesion_3\Resuelto> java -cp .
Error: Could not find or load main class Server
Caused by: java.lang.ClassNotFoundException: Server
PS D:\UNSA0.0\UNSA\septimosemestre\Sistemas-Distribuidos\LAB_SISTEMAS_DISTRIBUIDOS\sesion_3\src> cd ..
PS D:\UNSA0.0\UNSA\septimosemestre\Sistemas-Distribuidos\LAB_SISTEMAS_DISTRIBUIDOS\sesion_3\src> cd ..\Propuestas\
PS D:\UNSA0.0\UNSA\septimosemestre\Sistemas-Distribuidos\LAB_SISTEMAS_DISTRIBUIDOS\sesion_3\src\Propuestas> java -cp .
Server
Server listening on port 5001
Thread trying to create Object Input/Output Streams
23:10:47 UJMKJuandi just connected.

176 break;
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS D:\UNSA0.0\UNSA\septimosemestre\Sistemas-Distribuidos\LAB_SISTEMAS_DISTRIBUIDOS\sesion_3\src\Resuelto> java -cp . cliente
Hola cliente 0
PS D:\UNSA0.0\UNSA\septimosemestre\Sistemas-Distribuidos\LAB_SISTEMAS_DISTRIBUIDOS\sesion_3\src\Resuelto> java -cp . cliente
Hola cliente 1
PS D:\UNSA0.0\UNSA\septimosemestre\Sistemas-Distribuidos\LAB_SISTEMAS_DISTRIBUIDOS\sesion_3\src\Resuelto> java -cp . cliente
Hola cliente 2
PS D:\UNSA0.0\UNSA\septimosemestre\Sistemas-Distribuidos\LAB_SISTEMAS_DISTRIBUIDOS\sesion_3\src\Resuelto> cd ..
PS D:\UNSA0.0\UNSA\septimosemestre\Sistemas-Distribuidos\LAB_SISTEMAS_DISTRIBUIDOS\sesion_3\src\Propuestas> javac *.java
PS D:\UNSA0.0\UNSA\septimosemestre\Sistemas-Distribuidos\LAB_SISTEMAS_DISTRIBUIDOS\sesion_3\src\Propuestas> java -cp . Client
Enter server address (default: localhost): 5001
Enter port (default: 5001): 5001
Enter username: UJMKJuandi
Error connecting to server: java.net.SocketException: network is unreachable: connect
PS D:\UNSA0.0\UNSA\septimosemestre\Sistemas-Distribuidos\LAB_SISTEMAS_DISTRIBUIDOS\sesion_3\src\Propuestas> java -cp . Client
Enter server address (default: localhost): localhost
Enter port (default: 5001): 5001
Enter username: UJMKJuandi
Connection accepted localhost/127.0.0.1:5001
> |
  
```

Figura 2: Ejecución del ejemplo Cliente-Servidor

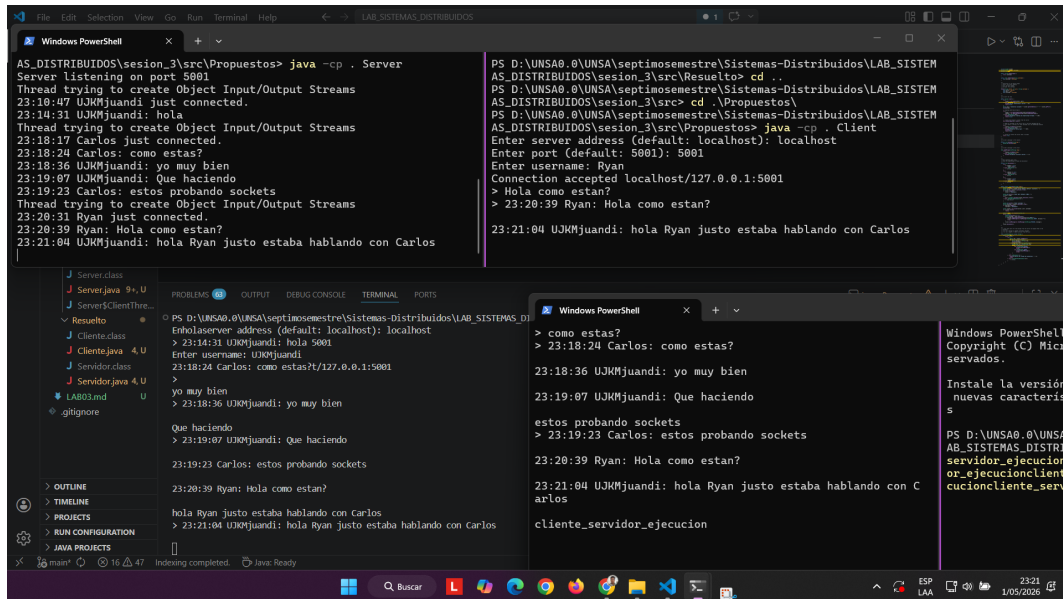


Figura 3: Ejecución de la Aplicación Chat Multicliente

5. CUESTIONARIO

5.1. 1. ¿Por qué y en qué momento usar sockets TCP y sockets UDP?

Respuesta:

Los sockets TCP (Transmission Control Protocol) y UDP (User Datagram Protocol) son dos protocolos de transporte fundamentales que ofrecen características diferentes según las necesidades de la aplicación.

5.1.1. Sockets TCP

Características:

- **Confiabilidad:** Garantiza que todos los datos se entreguen sin pérdida
- **Orientado a Conexión:** Establece una conexión antes de enviar datos
- **Orden:** Los datos se reciben en el mismo orden que se envían
- **Flujo Controlado:** Implementa control de congestión y flujo
- **Overhead:** Mayor consumo de recursos y ancho de banda

Cuándo usar:

1. Aplicaciones que requieren entrega garantizada (correo electrónico, transferencia de archivos)
2. Comunicación interactiva en tiempo real donde los datos deben ser correctos (navegación web, chat)
3. Cuando la exactitud es más importante que la velocidad
4. Transferencia de datos estructurados o mensajes complejos
5. Ejemplos: HTTP/HTTPS, SSH, Telnet, FTP, Bases de datos

5.1.2. Sockets UDP

Características:

- **Sin Conexión:** No requiere establecer una conexión previa
- **Sin Garantía:** Los paquetes pueden perderse sin notificación
- **Bajo Overhead:** Menor consumo de recursos y ancho de banda
- **Velocidad:** Latencia muy baja
- **Sin Orden:** Los paquetes pueden llegar fuera de orden

Cuándo usar:

1. Aplicaciones que requieren baja latencia (videojuegos, videoconferencia)
2. Streaming de audio/video en tiempo real
3. Aplicaciones DNS que necesitan respuestas rápidas
4. Cuando la velocidad es más crítica que la exactitud
5. Transmisiones de datos que toleran pequeñas pérdidas
6. Ejemplos: DNS, NTP, DHCP, VoIP, streaming de video

Conclusión: TCP se debe usar cuando es crítico que los datos se entreguen correctamente, mientras que UDP es ideal para aplicaciones donde la velocidad es más importante que la certeza de entrega.

5.2. 2. En un socket TCP, ¿cuándo sabe el servidor que el cliente ha cerrado la conexión?

Respuesta:

En un socket TCP, el servidor detecta que el cliente ha cerrado la conexión de varias maneras:

5.2.1. Cierre Normal del Socket

El servidor sabe que el cliente cerró la conexión cuando:

1. **EOF (End of File):** Al intentar leer datos, si recibe EOF (-1 en Java), significa que el cliente cerró la conexión de lectura

```
int data = inputStream.read();
if (data == -1) {
    // El cliente cerr la conexin
    System.out.println("Cliente desconectado");
}
```

2. **Excepción EOFException:** Al usar flujos de datos estructurados como `DataInputStream` o `ObjectInputStream`

```
try {
    String msg = dataInputStream.readUTF();
} catch (EOFException e) {
    // El cliente cerr la conexin
    System.out.println("Cliente desconectado abruptamente");
}
```

5.2.2. Métodos de Detección

En Java, existen varios métodos para detectar la desconexión:

- **socket.isClosed():** Verifica si el socket está cerrado
- **socket.isConnected():** Verifica si el socket está conectado (limitado)
- **Excepciones IOException:** Se lanzan cuando hay error en la comunicación

5.2.3. Código de Ejemplo

Listing 8: Detección de desconexión de cliente

```
1 ServerSocket serverSocket = new ServerSocket(5000);
2 Socket clientSocket = serverSocket.accept();
3 DataInputStream input = new DataInputStream(clientSocket.getInputStream());
4
5 while (true) {
6     try {
7         String message = input.readUTF();
8         System.out.println("Mensaje: " + message);
9     } catch (EOFException e) {
10        System.out.println("Cliente desconectado");
11        break;
12    } catch (IOException e) {
13        System.out.println("Error en la conexin: " + e.getMessage());
14        break;
15    }
16 }
17
18 clientSocket.close();
19 serverSocket.close();
```

Conclusión: El servidor detecta la desconexión del cliente principalmente a través de excepciones `EOFException` o `IOException` cuando intenta leer del socket, o al recibir un valor -1 al leer bytes individuales. Es responsabilidad del servidor manejar estas excepciones adecuadamente para limpiar recursos.

6. Conclusiones

6.1. Aprendizajes Clave

La realización de este laboratorio permitió consolidar los siguientes aprendizajes:

1. Conceptos Fundamentales de Sockets:

- Entender la arquitectura cliente-servidor en Java
- Dominar la creación y gestión de sockets TCP
- Comprender el ciclo de vida de una conexión TCP

2. Programación de Red en Java:

- Uso correcto de `Socket` y `ServerSocket`
- Manejo de flujos de entrada/salida para comunicación de red

- Serialización de objetos para intercambio de datos complejos

3. Aplicaciones Multicliente:

- Implementación de servidores que atienden múltiples clientes usando threads
- Sincronización para evitar condiciones de carrera
- Broadcast de mensajes entre múltiples clientes

4. Manejo de Errores en Red:

- Captura de excepciones de conexión (IOException, EOFException)
- Importancia del cierre adecuado de recursos
- Técnicas para detectar desconexiones

6.2. Observaciones

- Los sockets TCP son fundamentales para la comunicación distribuida confiable
- La programación multihilo es esencial para servidores que atienden múltiples clientes
- El manejo correcto de excepciones es crítico en aplicaciones de red
- La serialización de objetos facilita el intercambio de datos complejos entre sistemas distribuidos

6.3. Recomendaciones Futuras

1. Implementar un servidor persistente que no limite el número de clientes
2. Agregar autenticación de usuarios en la aplicación chat
3. Implementar un protocolo de sockets UDP para comparar con TCP
4. Añadir persistencia de mensajes (logs o base de datos)
5. Implementar SSL/TLS para comunicación segura entre cliente y servidor
6. Explorar frameworks como Netty para aplicaciones de red más complejas

7. REFERENCIAS Y BIBLIOGRAFÍA

1. Tanenbaum, A.S. (2008). *Sistemas distribuidos: principios y paradigmas*. México. Pearson Educación.
2. Ceballos, F. J. (2006). *Java 2, Curso de programación*. México: Alfaomega, RaMa.
3. Deitel, H. M., & Deitel, P. J. (2004). *Cómo programar en Java*. México: Pearson Educación.
4. García Tomás, J., Ferrando, S., & Piattini, M. (2001). *Redes para procesos distribuidos*. México: Alfaomega Ra-Ma.
5. Orfali, R., & Harkey, D. (1998). *Client/Server Programming with Java and CORBA*. USA: Wiley.
6. Oracle. (2024). *Java Networking and Proxies*. Documentación oficial de Java. <https://docs.oracle.com/javase/tutorial/networking/>